

Day 34 – Docker Compose Advanced

1. Create Project

```
None
mkdir day34-compose
cd day34-compose

mkdir app
```

2. Create Flask App

app/app.py

```
None
from flask import Flask
import psycopg2
import os

app = Flask(__name__)

@app.route("/")
def home():
    conn = psycopg2.connect(
        host=os.getenv("DB_HOST"),
        database=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )
    conn.close()
    return "Connected to PostgreSQL!"

app.run(host="0.0.0.0", port=5000)
```

app/requirements.txt

```
None
Flask
psycpg2-binary
```

3. Create Dockerfile

app/Dockerfile

```
None
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

4. Create docker-compose.yml

```
None
version: "3.9"

services:

  web:
    build: ./app
    ports:
      - "5000:5000"
    environment:
```

```
    DB_HOST: db
    DB_NAME: mydb
    DB_USER: postgres
    DB_PASSWORD: password
depends_on:
  db:
    condition: service_healthy
networks:
  - app-network
labels:
  project: day34

db:
  image: postgres:16
  restart: always
  environment:
    POSTGRES_DB: mydb
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
  volumes:
    - postgres-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres -d mydb"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - app-network
  labels:
    project: day34

redis:
  image: redis:7
  networks:
    - app-network
  labels:
```

project: day34

networks:

app-network:

volumes:

postgres-data:

Task 1

Build everything

None

`docker compose up --build`

Visit

None

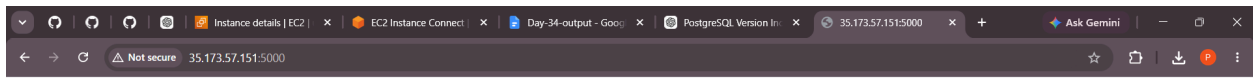
`http://EC2_PUBLIC_IP:5000`

```
aws | Search | [Alt+S] | Ask Amazon Q | United States (N. Virginia) | Ashirwad Tours & Travels (769369611777) | Abhijeet
redis-1 | 1:M 20 Jul 2026 14:30:30.571 * Increased maximum number of open files to 10032 (it was originally set to 1024).
redis-1 | 1:M 20 Jul 2026 14:30:30.572 * monotonic clock: POSIX clock_gettime
redis-1 | 1:M 20 Jul 2026 14:30:30.573 * Running mode=standalone, port=6379.
redis-1 | 1:M 20 Jul 2026 14:30:30.575 * Server initialized
redis-1 | 1:M 20 Jul 2026 14:30:30.576 * Loading RDB produced by version 7.4.9
redis-1 | 1:M 20 Jul 2026 14:30:30.576 * RDB age 24 seconds
redis-1 | 1:M 20 Jul 2026 14:30:30.577 * RDB memory usage when created 0.90 Mb
redis-1 | 1:M 20 Jul 2026 14:30:30.577 * Done loading RDB, keys loaded: 0, keys expired: 0.
redis-1 | 1:M 20 Jul 2026 14:30:30.578 * DB loaded from disk: 0.002 seconds
redis-1 | 1:M 20 Jul 2026 14:30:30.578 * Ready to accept connections tcp
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
db-1 |
db-1 | 2026-07-20 14:30:30.616 UTC [1] LOG: starting PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2026-07-20 14:30:30.616 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-07-20 14:30:30.617 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2026-07-20 14:30:30.625 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-07-20 14:30:30.634 UTC [28] LOG: database system was shut down at 2026-07-20 14:30:16 UTC
db-1 | 2026-07-20 14:30:30.648 UTC [1] LOG: database system is ready to accept connections
web-1 | * Serving Flask app 'app'
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.18.0.4:5000
web-1 | Press CTRL+C to quit
web-1 | 115.96.217.121 - - [20/Jul/2026 14:31:05] "GET / HTTP/1.1" 200 -
web-1 | 115.96.217.121 - - [20/Jul/2026 14:31:08] "GET /favicon.ico HTTP/1.1" 404 -
Enable Watch
```

Output

None

Connected to PostgreSQL!



Task 2

Stop everything

None

`docker compose down`

Start again

None

`docker compose up`

Observe logs.

The **web** service waits until PostgreSQL healthcheck passes.

Task 3

Find database container

None

```
docker ps
```

Kill it

None

```
docker kill <db_container_id>
```

Watch

None

```
docker ps
```

Because

None

```
restart: always
```

Docker automatically starts it again.

Now change

None

```
restart: always
```

to

None

```
restart: on-failure
```

Run

None

```
docker compose down  
docker compose up
```

Notes

- `always` → Restart even after manual stop or daemon restart.
 - `on-failure` → Restart only if the container exits with a non-zero status.
-

Task 4

Modify

None

```
return "Hello Day 34!"
```

Rebuild

None

```
docker compose up --build
```

Only one command needed.

Task 5

Already completed using

- Named network

None

```
networks:  
  app-network:
```

- Named volume

None

```
volumes:
  postgres-data:
```

- Labels

None

```
labels:
  project: day34
```

Task 6 (Bonus)

Scale

None

```
docker compose up --scale web=3
```

```
ubuntu@ip-172-31-73-72:~$ cd day34-compose/
ubuntu@ip-172-31-73-72:~/day34-compose$ docker compose up --scale web=3
WARN[0000] /home/ubuntu/day34-compose/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
   ✓ Container day34-compose-db-1   Running      0.0s
   ✓ Container day34-compose-web-3   Created      0.1s
   ✓ Container day34-compose-web-2   Created      0.1s
Attaching to db-1, redis-1, web-1, web-2, web-3
redis-1 | 1:c 20 Jul 2026 16:11:40.991 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Background saving disabled, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
redis-1 | 1:c 20 Jul 2026 16:11:40.992 # WARNING Your system is configured to use the 'xen' clocksource which might lead to degraded performance. Check the result of the [slow-clocksources] system check: run 'redis-server --check-system' to check if the system's clocksource isn't degrading performance.
redis-1 | 1:c 20 Jul 2026 16:11:40.992 * oOoOoOoOoOoOo Redis is starting oOoOoOoOoOoOo
redis-1 | 1:c 20 Jul 2026 16:11:40.992 * Redis version=7.4.9, bits=64, commit=00000000, modified=0, pid=1, just started
redis-1 | 1:c 20 Jul 2026 16:11:40.992 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
redis-1 | 1:m 20 Jul 2026 16:11:40.992 * Increased maximum number of open files to 10032 (it was originally set to 1024).
redis-1 | 1:m 20 Jul 2026 16:11:40.992 * monotonic clock: POSIX clock_gettime
redis-1 | 1:m 20 Jul 2026 16:11:41.001 * Running mode=standalone, port=6379.
redis-1 | 1:m 20 Jul 2026 16:11:41.002 * Server initialized
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * Loading RDB produced by version 7.4.9
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * RDB age 5307 seconds
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * RDB memory usage when created 0.90 MB
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * Done loading RDB, Keys loaded: 0, keys expired: 0.
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * DB loaded from disk: 0.002 seconds
redis-1 | 1:m 20 Jul 2026 16:11:41.005 * Ready to accept connections tcp
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint day34-compose-web-3 (417b2d8e2df419a35fb4e2bf07f81cea7bffa5abd142256378c78c5988c60f0): Bind for :::5000 failed: port is already allocated
ubuntu@ip-172-31-73-72:~/day34-compose$
```

You'll see an error similar to:

None

```
port is already allocated
```

or only one container can bind to:

None

5000:5000

Notes

Simple scaling doesn't work with fixed host port mappings because multiple containers cannot bind to the same host port. In production, this is usually handled by a reverse proxy or load balancer (such as Nginx or Traefik), which routes traffic to multiple application containers.

Files to Submit

None

```
day34-compose/  
|  
├─ docker-compose.yml  
|  
├─ app/  
|   ├─ Dockerfile  
|   ├─ app.py  
|   └─ requirements.txt  
└─ day-34-compose-advanced.md
```

This setup covers all the required tasks: a custom-built web app, PostgreSQL, Redis, `depends_on` with health checks, restart policies, named networks and volumes, labels, and the scaling experiment.